

Ensembles & Régularisation

MIASHS M1 - Semestre 1

Table des matières

1. Méthodes ensemblistes	1
2. Régularisation	23

1. Méthodes ensemblistes

1.1. Introduction

Les méthodes ensemblistes constituent l'une des approches les plus efficaces en apprentissage automatique. L'idée fondamentale est simple : **combinaison de plusieurs modèles pour obtenir de meilleures performances qu'avec un modèle unique.**

1.1.1. Cadre théorique

Soit $\mathcal{X}$ notre espace d'entrée et $\mathcal{Y}$ notre espace de sortie. Soit $X, Y \in \mathcal{X} \times \mathcal{Y}$ deux variables aléatoires et soit $\mathbb{P}$ leur mesure jointe. Notre objectif est de trouver une application $h : \mathcal{X} \rightarrow \mathcal{Y}$ qui minimise une certaine erreur qu'on notera L .

N'ayant pas accès aux variables aléatoires X et Y , nous collectons un jeu de données $S_n = \{(X_i, Y_i)\}_{i \leq n} \sim \mathbb{P}^n$ et nous construisons un risque empirique :

$$L_n(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(X_i), Y_i)$$

où ℓ définit une erreur élémentaire pour une unique prédiction.

1.1.2. Motivation théorique

Théorème 4.1 (Réduction de variance par agrégation)

Considérons une famille de prédicteurs $\{y_j\}_{j \leq m}$ et leur agrégation :

$$\hat{y} = \frac{1}{m} \sum_{j=1}^m y_j(x)$$

Si chaque prédicteur vérifie $\hat{y}_{ij} = y(x_j) + \varepsilon_{ij}$ où ε_{ij} est un bruit **centré** : $\mathbb{E}[\varepsilon_{ij}] = 0$, alors :

$$\mathbb{E} \left[\left(y(X) - \frac{1}{m} \sum_{j=1}^m \hat{y}_j(X) \right)^2 \right] = \mathbb{E} \left[\left(\frac{1}{m} \sum_{j=1}^m \varepsilon_j \right)^2 \right]$$

Si de plus les erreurs sont **indépendantes** : $\mathbb{E}[\varepsilon_j \varepsilon_k] = 0$ pour $j \neq k$, alors :

$$\mathbb{E} \left[\left(\frac{1}{m} \sum_{j=1}^m \varepsilon_j \right)^2 \right] = \frac{1}{m} \mathbb{E}[\varepsilon^2]$$

La variance est **réduite d'un facteur m** !

Remarque cruciale : Cette réduction de variance n'est effective que si les modèles font des erreurs **complémentaires**. Si tous les

modèles se trompent de la même manière, l'agrégation n'apporte rien.

1.2. L'approche naïve

L'approche la plus simple pour combiner plusieurs modèles consiste à moyenner leurs prédictions.

1.2.1. Agrégation par vote majoritaire

Définition 4.2 (Vote majoritaire)

Pour un problème de classification avec m modèles $\{h_j\}_{j=1}^m$, la prédiction finale est :

$$\hat{y}(x) = \arg \max_{c \in \mathcal{Y}} \sum_{j=1}^m \mathbb{1}[h_j(x) = c]$$

où $\mathbb{1}[\cdot]$ est la fonction indicatrice.

Exercice 4.1

Implémentez une classe `MajorityVoting` qui combine plusieurs classifieurs par vote majoritaire. La méthode `predict` doit retourner la classe ayant obtenu le plus de votes.

```
class MajorityVoting:
    def __init__(self, *models):
        self.models = models

    def predict(self, X):
        # À compléter
        pass
```

1.2.2. Agrégation par moyenne

Définition 4.3 (Moyenne pour la régression)

Pour un problème de régression avec m modèles $\{h_j\}_{j=1}^m$, la prédiction finale est :

$$\hat{y}(x) = \frac{1}{m} \sum_{j=1}^m h_j(x)$$

Exercice 4.2

Implémentez une classe `AverageVoting` qui combine plusieurs régresseurs par moyenne arithmétique.

1.2.3. Conditions de succès

L'agrégation naïve fonctionne bien si :

1. **Diversité** : Les modèles font des erreurs différentes
2. **Qualité individuelle** : Chaque modèle est meilleur que le hasard
3. **Indépendance** : Les erreurs sont décorrélées

Attention : Si tous les modèles se trompent sur les mêmes exemples, l'agrégation n'améliore rien !

1.3. Bayesian Model Averaging (BMA)

1.3.1. Principe théorique

L'approche bayésienne permet de pondérer les modèles selon leur vraisemblance a posteriori.

Définition 4.4 (Bayesian Model Averaging)

Soit une famille de M modèles probabilistes $\{\mathcal{M}_j\}_{j=1}^M$. Pour une nouvelle observation x , la prédiction BMA est :

$$p(y|x, \mathcal{D}) = \sum_{j=1}^M p(y|x, \mathcal{M}_j, \mathcal{D})p(\mathcal{M}_j | \mathcal{D})$$

où :

- $p(y|x, \mathcal{M}_j, \mathcal{D})$: prédiction du modèle j conditionnellement aux données
- $p(\mathcal{M}_j | \mathcal{D})$: probabilité a posteriori du modèle j

1.3.2. Calcul des poids a posteriori

La probabilité a posteriori de chaque modèle se calcule via la règle de Bayes :

$$p(\mathcal{M}_j | \mathcal{D}) = \frac{p(\mathcal{D} | \mathcal{M}_j)p(\mathcal{M}_j)}{\sum_{k=1}^M p(\mathcal{D} | \mathcal{M}_k)p(\mathcal{M}_k)}$$

où :

- $p(\mathcal{D} | \mathcal{M}_j)$: vraisemblance des données sous le modèle j
- $p(\mathcal{M}_j)$: probabilité a priori du modèle j (souvent uniforme)

1.3.3. Implémentation pratique

En pratique, on travaille souvent avec les log-probabilités pour éviter les problèmes numériques :

$$\begin{aligned}\log p(\mathcal{M}_j \mid \mathcal{D}) &= \log p(\mathcal{D} \mid \mathcal{M}_j) \\ &\quad + \log p(\mathcal{M}_j) - \log \sum_{k=1}^M e^{\log p(\mathcal{D} \mid \mathcal{M}_k) + \log p(\mathcal{M}_k)}\end{aligned}$$

La fonction `logsumexp` permet de calculer $\log \sum e$ en évitant les problèmes numériques.

Exercice 4.3

Implémentez une classe `BayesianModelAveraging` pour la classification. Utilisez la log-vraisemblance des prédictions pour calculer les poids a posteriori.

Indice : Pour un modèle de classification probabiliste, $\log p(\mathcal{D} \mid \mathcal{M}_j) = \sum_{i=1}^n \log p(y_i \mid x_i, \mathcal{M}_j)$

1.3.4. Avantages et limitations

Avantages du BMA

- ✓ Prise en compte rigoureuse de l'incertitude sur les modèles
- ✓ Poids adaptatifs selon les performances
- ✓ Fondement théorique solide

Limitations

- × Nécessite des modèles probabilistes
- × Sensible au choix des *a priori*

1.4. Bagging (Bootstrap Aggregating)

1.4.1. Principe du Bootstrap

La difficulté principale avec l'agrégation naïve est que pour une classe de modèles donnée, un même jeu d'apprentissage produit la même solution. Le **bagging** résout ce problème en créant de la variabilité via des échantillons bootstrap.

Définition 4.5 (Échantillon Bootstrap)

Soit $S_n = \{(X_i, Y_i)\}_{i=1}^n$ un jeu de données de taille n . Un échantillon bootstrap S_n^* est obtenu en tirant n points **aléatoirement avec remise** dans S_n .

Propriétés :

- Certains points apparaissent plusieurs fois
- Environ $(1 - \frac{1}{e}) \approx 63.2\%$ des points originaux sont présents
- Environ $\frac{1}{e} \approx 36.8\%$ des points sont absents

Exercice 4.2 (Démonstration des propriétés du bootstrap)

Démontrez rigoureusement les propriétés probabilistes énoncées dans la Définition 4.5 :

Partie 1 : Probabilité qu'un point soit absent

1. Quelle est la probabilité qu'un point donné (X_i, Y_i) ne soit **pas** sélectionné lors d'un tirage ?
2. Quelle est la probabilité qu'il ne soit sélectionné lors d'**aucun** des n tirages ?

3. Calculez $\lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right)^n$ et montrez que cela donne $\frac{1}{e}$.

Partie 2 : Probabilité qu'un point soit présent

4. Déduisez la probabilité qu'un point donné soit présent **au moins une fois** dans l'échantillon bootstrap.

5. Vérifiez numériquement pour $n = 10, 100, 1000$ que cette probabilité converge bien vers $1 - \frac{1}{e} \approx 0.632$.

Partie 3 : Distribution du nombre d'occurrences

6. Soit N_i le nombre de fois qu'apparaît le point (X_i, Y_i) dans l'échantillon bootstrap. Quelle est la loi de N_i ?

7. Calculez $\mathbb{E}[N_i]$ et $\text{Var}(N_i)$.

1.4.2. Algorithme du Bagging

Algorithme 4.1 (Bagging)

Entrée : Jeu de données S_n , algorithme d'apprentissage $\mathcal{A}$, nombre de modèles M

Pour $j = 1, \dots, M$:

1. Générer un échantillon bootstrap $S_n^{(j)}$ à partir de S_n

2. Entraîner un modèle $h_j = \mathcal{A}(S_n^{(j)})$

Sortie : Prédiction agrégée $\hat{h}(x) = \frac{1}{M} \sum_{j=1}^M h_j(x)$ (régression)
ou vote majoritaire (classification)

1.4.3. Analyse théorique

Théorème 4.2 (Réduction de variance par bagging)

Si les modèles de base ont une variance σ^2 et sont décorrélés, alors le modèle baggé a une variance $\frac{\sigma^2}{M}$.

En pratique, les modèles ne sont pas totalement décorrélés, mais le bagging réduit significativement la variance pour les modèles instables (arbres de décision, réseaux de neurones).

Intuition : Le bagging stabilise les algorithmes **instables** (qui changent beaucoup avec de petites variations des données) mais n'aide pas les algorithmes **stables** (comme la régression linéaire).

1.4.4. Out-of-Bag (OOB) Error

Définition 4.6 (Out-of-Bag Error)

Pour chaque exemple (x_i, y_i) , on peut évaluer la prédiction en utilisant seulement les modèles pour lesquels cet exemple n'était **pas** dans l'échantillon bootstrap correspondant.

L'erreur OOB fournit une estimation non biaisée de l'erreur de généralisation **sans ensemble de validation séparé**.

1.4.5. Exemple pratique avec scikit-learn

```
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
```

```
# Modèle de base instable
```

```
base_model = DecisionTreeRegressor()
```

```
# Bagging avec 100 estimateurs
```

```
bagging_model = BaggingRegressor(  
    base_estimator=base_model,  
    n_estimators=100,  
    random_state=42  
)
```

Exercice 4.4

Comparez les performances d'un arbre de décision seul vs un modèle baggé avec 100 arbres sur un jeu de données de votre choix. Observez la différence de variance des prédictions.

1.5. Random Forest

1.5.1. Motivation

Les forêts aléatoires combinent le bagging avec une technique supplémentaire pour créer encore plus de diversité : la **sélection aléatoire de features**.

Problème du bagging pur

Même avec des échantillons bootstrap différents, les arbres peuvent être très corrélés si :

- Quelques features sont très prédictives
- Ces features dominent toujours les divisions aux nœuds racines
- Les arbres ont des structures similaires

1.5.2. Algorithme Random Forest

Algorithme 4.2 (Random Forest)

Paramètres : M = nombre d'arbres, m = nombre de features à considérer par division

Pour $j = 1, \dots, M$:

1. Générer un échantillon bootstrap $S_n^{(j)}$
2. Construire un arbre T_j en appliquant la modification suivante :
 - À chaque nœud, sélectionner aléatoirement m features parmi les d disponibles
 - Choisir la meilleure division parmi ces m features seulement

Sortie : Prédiction agrégée des M arbres

1.5.3. Choix des hyperparamètres

Règles empiriques pour m :

- **Classification :** $m = \sqrt{d}$
- **Régression :** $m = \frac{d}{3}$

où d est le nombre total de features.

Ces valeurs offrent un bon compromis entre diversité et qualité des divisions individuelles.

1.5.4. Avantages des Random Forest

Avantages

- ✓ Excellentes performances prêtes à l'emploi
- ✓ Robuste au surajustement
- ✓ Gestion naturelle des variables catégorielles et numériques
- ✓ Estimation de l'importance des variables (cf. cours Classification supervisée)
- ✓ Estimation OOB de l'erreur de généralisation
- ✓ Parallélisation naturelle

Exercice 4.5

Utilisez `RandomForestRegressor` de scikit-learn sur un jeu de données et analysez l'importance des variables. Comparez les résultats avec un modèle baggé simple.

1.6. Boosting

1.6.1. Principe général

Contrairement au bagging qui entraîne les modèles en **parallèle**, le boosting les entraîne de manière **séquentielle**, chaque nouveau modèle essayant de corriger les erreurs des précédents.

Philosophie du Boosting

- **Bagging** : « Entraîner plusieurs experts indépendants »
- **Boosting** : « Entraîner une séquence d'experts qui apprennent de leurs erreurs »

1.6.2. AdaBoost (Adaptive Boosting)

AdaBoost est l'algorithme de boosting le plus classique pour la classification binaire.

Algorithme 4.3 (AdaBoost)

Entrée : $S = \{(x_i, y_i)\}_{i=1}^n$ avec $y_i \in \{-1, +1\}$, nombre d'itérations T

Initialisation : $w_i^1 = \frac{1}{n}$ pour $i = 1, \dots, n$

Pour $t = 1, \dots, T$:

1. Entraîner un classificateur faible h_t sur (S, w^t)
2. Calculer l'erreur pondérée : $\varepsilon_t = \sum_{i: h_t(x_i) \neq y_i} w_i^t$
3. Si $\varepsilon_t \geq \frac{1}{2}$: arrêter
4. Calculer le poids du modèle : $\alpha_t = \frac{1}{2} \log\left(\frac{1-\varepsilon_t}{\varepsilon_t}\right)$
5. Mettre à jour les poids des exemples :

$$w_i^{t+1} = \frac{w_i^t}{Z_t} e^{-\alpha_t y_i h_t(x_i)}$$

où Z_t est une constante de normalisation

Sortie : $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$

1.6.3. Interprétation de l'algorithme

1. **Poids des exemples** : Les exemples mal classifiés voient leur poids augmenter
2. **Poids des modèles** : α_t est grand si ε_t est petit (bon modèle)
3. **Focus adaptatif** : Chaque nouveau modèle se concentre sur les erreurs des précédents

1.6.4. Propriétés théoriques remarquables

Théorème 4.3 (Borne sur l'erreur d'entraînement)

L'erreur d'entraînement d'AdaBoost après T itérations est bornée par :

$$\text{Err}_{\text{train}} \leq \prod_{t=1}^T 2\sqrt{\varepsilon_t(1-\varepsilon_t)} = \prod_{t=1}^T \sqrt{1-4\gamma_t^2}$$

où $\gamma_t = \frac{1}{2} - \varepsilon_t$ est l'avantage du classifieur faible t sur le hasard.

Conséquence importante : Si chaque classifieur faible a un avantage $\gamma > 0$ sur le hasard, l'erreur d'entraînement décroît **exponentiellement** !

1.6.5. Lien fondamental avec la fonction de perte exponentielle

L'efficacité remarquable d'AdaBoost trouve son explication théorique dans sa connexion avec la minimisation d'une fonction de perte particulière.

Théorème 4.4 (AdaBoost et perte exponentielle)

L'algorithme AdaBoost peut être vu comme une méthode de **descente de coordonnées** pour minimiser la fonction de perte exponentielle :

$$L_{\text{exp}}(F) = \frac{1}{n} \sum_{i=1}^n e^{-y_i F(x_i)}$$

où $F(x) = \sum_{t=1}^T \alpha_t h_{t(x)}$ est la fonction de décision finale.

1.6.6. Dérivation des formules d'AdaBoost

Cette connexion permet de comprendre pourquoi les formules d'AdaBoost sont optimales :

Démonstration (esquisse)

Étape 1 : À l'itération t , on a $F_{t-1}(x) = \sum_{j=1}^{t-1} \alpha_j h_j(x)$

Étape 2 : On cherche (α_t, h_t) qui minimisent :

$$\sum_{i=1}^n e^{-y_i (F_{t-1}(x_i) + \alpha_t h_t(x_i))}$$

Étape 3 : En posant $w_i^{(t)} = e^{-y_i F_{t-1}(x_i)}$, ceci devient :

$$\sum_{i=1}^n w_i^{(t)} e^{-y_i \alpha_t h_t(x_i)}$$

Étape 4 : La minimisation par rapport à h_t donne exactement le critère d'erreur pondérée d'AdaBoost !

1.6.7. Interprétation en termes de margin

La perte exponentielle a une interprétation géométrique élégante :

Définition 4.7 (Margin)

Le **margin** d'un exemple (x_i, y_i) par rapport à un classificateur F est :

$$\text{margin}_i = y_i F(x_i)$$

Interprétation :

- Si $\text{margin}_i > 0$: classification correcte avec confiance $|\text{margin}_i|$

- Si $\text{margin}_i < 0$: classification incorrecte avec confiance $|\text{margin}_i|$
- Si $\text{margin}_i = 0$: indécision (frontière de décision)

La perte exponentielle $e^{-\text{margin}_i}$ pénalise **exponentiellement** les exemples avec une *marge* très négative :

$$L_{\text{exp}}(\text{margin}_i) = e^{-\text{margin}_i} = \begin{cases} e^{-\text{margin}_i} \rightarrow 0 & \text{si margin grand et positif} \\ 1 & \text{si margin nul} \\ e^{|\text{margin}_i|} \rightarrow +\infty & \text{si margin grand et négatif} \end{cases}$$

Propriété clé (Margin maximization)

AdaBoost tend à **maximiser le margin minimum** des exemples d'entraînement, conduisant à une meilleure généralisation.

Plus précisément, si les classificateurs faibles ont une marge strictement positive, AdaBoost converge vers un classificateur avec margin minimum maximal.

1.6.8. Comparaison avec la perte logistique

Il existe une connexion profonde entre AdaBoost et la régression logistique :

Analogie avec la régression logistique

La perte logistique est : $L_{\text{logistic}}(\text{margin}) = \log(1 + e^{-\text{margin}})$

Comparaison :

- **Perte exponentielle** : $L_{\text{exp}}(\text{margin}) = e^{-\text{margin}}$
- **Perte logistique** : $L_{\text{logistic}}(\text{margin}) = \log(1 + e^{-\text{margin}})$

Les deux fonctions sont **convexes** et pénalisent les faibles marges, mais la perte exponentielle est plus **agressive** pour les exemples très mal classifiés.

Cette agressivité explique à la fois la puissance d'AdaBoost et sa sensibilité au bruit et aux outliers.

1.6.9. Implications pratiques

Conséquences pour la pratique

Avantages de la perte exponentielle :

- ✓ Convergence rapide vers de faibles erreurs d'entraînement
- ✓ Maximisation automatique des marges
- ✓ Fondement théorique solide pour les formules d'AdaBoost

Inconvénients :

- × Sensibilité aux outliers (pénalité exponentielle)
- × Risque de surajustement sur données bruitées
- × Pas de régularisation intrinsèque

1.6.10. Extension aux pertes robustes

Pour pallier la sensibilité aux outliers, des variantes d'AdaBoost utilisent des pertes plus robustes :

- **LogitBoost** : Utilise la perte logistique (plus robuste)
- **AdaBoost.M2** : Perte modifiée pour les problèmes multi-classes
- **BrownBoost** : Perte adaptative qui « ignore » les exemples difficiles

1.6.11. Gradient Boosting

Une généralisation moderne du boosting consiste à voir le problème comme une **descente de gradient dans l'espace des fonctions**.

Principe du Gradient Boosting

1. Commencer avec un modèle initial F_0
2. À chaque étape m :
 - Calculer les résidus : $r_{i,m} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$
 - Entraîner un modèle h_m à prédire ces résidus
 - Mettre à jour : $F_m = F_{m-1} + \nu h_m$ (avec $\nu = \text{learning rate}$)

Avantage : S'applique à toute fonction de perte différentiable (régression, classification, ranking, etc.)

1.6.12. Implémentation pratique

```
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingRegressor
from sklearn.tree import DecisionTreeClassifier
```

```
# AdaBoost avec des stumps (arbres de profondeur 1)
ada = AdaBoostClassifier(
    base_estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=100,
    learning_rate=1.0
)
```

```
# Gradient Boosting pour la régression
gbr = GradientBoostingRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3
)
```

Exercice 4.6

1. Implémentez AdaBoost « à la main » avec des stumps comme classificateurs faibles
2. Comparez les performances avec `AdaBoostClassifier` de scikit-learn
3. Observez l'évolution de l'erreur d'entraînement et de test au cours des itérations

1.7. Méthodes modernes : XGBoost et LightGBM

1.7.1. Limitations du Gradient Boosting classique

- **Lenteur** : Construction séquentielle des arbres
- **Mémoire** : Stockage de tous les arbres
- **Overfitting** : Tendance au surajustement sans régularisation

1.7.2. XGBoost (eXtreme Gradient Boosting)

XGBoost améliore le gradient boosting traditionnel par :

Innovations de XGBoost

- ✓ **Régularisation L1/L2** sur les poids des feuilles
- ✓ **Approximation d'ordre 2** (utilisation de la hessienne)
- ✓ **Gestion des valeurs manquantes** native
- ✓ **Parallélisation** de la construction des arbres
- ✓ **Pré-tri des features** pour l'efficacité

1.7.3. LightGBM et CatBoost

- **LightGBM** : Croissance des arbres en largeur d'abord (**leaf-wise**) au lieu de niveau par niveau
- **CatBoost** : Gestion native des variables catégorielles sans pré-traitement

1.7.4. Hyperparamètres critiques

Paramètre	Description	Valeurs typiques
n_estimators	Nombre d'arbres	100-1000
learning_rate	Taux d'apprentissage	0.01-0.3
max_depth	Profondeur max des arbres	3-8
subsample	Fraction d'exemples par arbre	0.8-1.0
colsample_bytree	Fraction de features par arbre	0.8-1.0
reg_alpha	Régularisation L1	0-10
reg_lambda	Régularisation L2	1-10

1.8. Synthèse et comparaison des méthodes

1.8.1. Tableau comparatif

Méthode	Parallélisation	Stabilité	Interprétabilité	Performance
Vote majoritaire	✓	++	+++	+
Bagging	✓	++	++	++
Random Forest	✓	++	++	+++
AdaBoost	×	+	+	++
Gradient Boosting	×	+	+	+++
XGBoost	Partiel	+	+	++++

1.8.2. Guide de choix

Quand utiliser quoi ?

Random Forest :

- Premier choix pour un modèle robuste et interprétable
- Données mixtes (catégorielles + numériques)
- Besoin d'importance des variables

XGBoost/LightGBM :

- Compétitions de machine learning
- Optimisation fine des performances
- Grands jeux de données

Bagging :

- Modèles de base instables (arbres profonds, réseaux de neurones)
- Réduction de variance

AdaBoost :

- Modèles de base simples (stumps)
- Classification binaire
- Données avec structure séquentielle

1.8.3. Bonnes pratiques

Recommandations pratiques

1. **Commencer simple** : Random Forest avec paramètres par défaut
2. **Valider rigoureusement** : Cross-validation pour éviter l'overfitting

3. **Diversifier les modèles de base** : Différents algorithmes, hyperparamètres
4. **Surveiller la complexité** : Plus de modèles ne garantit pas une amélioration
5. **Exploiter l'OOB** : Estimation gratuite de l'erreur de généralisation

Projet final

Implémentez et comparez sur un jeu de données réel :

1. Un modèle Random Forest
2. Un modèle XGBoost
3. Un ensemble combinant différents types d'algorithmes

Analysez les trade-offs performance/complexité/interprétabilité.

1.9. Conclusion

Les méthodes ensemblistes illustrent parfaitement le principe que « **l'union fait la force** » en machine learning.

Les points clés à retenir :

- **Diversité** : La clé du succès est la complémentarité des erreurs
- **Stabilisation** : Réduction de variance pour les modèles instables
- **Réduction de biais** : Amélioration des modèles faibles par boosting
- **Compromis** : Performance vs interprétabilité vs complexité computationnelle

- **Évolution** : Des méthodes simples (vote) aux algorithmes optimisés (XGBoost)

2. Régularisation

2.1. Introduction à la régularisation

La régularisation est une technique fondamentale en apprentissage statistique qui vise à **contrôler la complexité** des modèles pour améliorer leur capacité de généralisation. Elle constitue une approche pour lutter contre le surajustement (overfitting).

2.1.1. Cadre théorique général

Problème d'optimisation régularisé

Au lieu de minimiser simplement le risque empirique :

$$\hat{\theta} = \arg \min(\theta) L_{n(\theta)} = \arg \min(\theta) \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta(x_i)}, y_i)$$

On introduit un terme de régularisation :

$$\hat{\theta}_{\lambda} = \arg \min(\theta) L_{n(\theta)} + \lambda \Omega(\theta)$$

où :

- $\Omega(\theta)$: terme de régularisation (pénalité de complexité)
- $\lambda \geq 0$: paramètre de régularisation (force de la pénalité)

2.1.2. Motivation théorique

Le terme de régularisation agit comme un **a priori** sur les paramètres du modèle :

- $\lambda = 0$: Pas de régularisation (risque de surajustement)
- λ **petit** : Régularisation faible (favorise l'ajustement aux données)
- λ **grand** : Régularisation forte (favorise la simplicité)

Théorème 5.1 (Décomposition biais-variance avec régularisation)

Pour un modèle régularisé, l'erreur de généralisation se décompose en :

$$\mathbb{E}\left[\left(y - \hat{f}_\lambda(x)\right)^2\right] = \text{Biais}^2\left[\hat{f}_\lambda(x)\right] + \text{Var}\left[\hat{f}_\lambda(x)\right] + \sigma^2$$

La régularisation introduit un **biais** mais **réduit la variance**, créant un trade-off plus intéressant.

2.2. Régularisation L2 (Ridge)

2.2.1. Définition et formulation

Définition 5.1 (Régularisation L2 - Ridge)

Pour un modèle linéaire $f_{\theta(x)} = \theta^\top x$, la régularisation Ridge ajoute une pénalité quadratique :

$$\hat{\theta}_{\text{Ridge}} = \arg \min(\theta) \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2$$

où $\|\theta\|_2^2 = \sum_{j=1}^p \theta_j^2$ est la norme euclidienne au carré.

2.2.2. Solution analytique

La régularisation L2 préserve la convexité du problème et admet une solution analytique (cf. cours d'optimisation) :

$$\hat{\theta}_{\text{Ridge}} = (X^\top X + \lambda I)^{-1} X^\top y$$

Cette formule révèle plusieurs propriétés importantes :

Propriétés de stabilisation de Ridge

1. **Conditionnement amélioré** : L'ajout de λI améliore le conditionnement de $X^\top X$
2. **Inversion garantie** : $(X^\top X + \lambda I)$ est toujours inversible pour $\lambda > 0$
3. **Stabilité numérique** : Réduction des problèmes de multicollinéarité
4. **Réduction de variance** : Les coefficients sont « contractés » vers zéro

2.2.3. Interprétation géométrique

La régularisation L2 peut être vue comme une **projection** des coefficients non régularisés sur une boule euclidienne :

```
import matplotlib.pyplot as plt
import numpy as np

# Visualisation de la contrainte L2 dans le plan (theta1, theta2)
theta = np.linspace(-2, 2, 100)
theta1, theta2 = np.meshgrid(theta, theta)

# Contrainte L2 : theta1^2 + theta2^2 <= t
circle = theta1**2 + theta2**2
```

2.2.4. Effet de rétrécissement (shrinkage)

Proposition 5.1 (Facteurs de rétrécissement)

Soit $X = U\Sigma V^\top$ la décomposition SVD de la matrice de design. Les coefficients Ridge s'écrivent :

$$\hat{\theta}_{\text{Ridge}} = \sum_{j=1}^r \frac{\sigma_j^2}{\sigma_j^2 + \lambda} v_j (u_j^\top y)$$

Les facteurs $\frac{\sigma_j^2}{\sigma_j^2 + \lambda}$ montrent comment Ridge **rétrécit** plus fortement les directions associées aux petites valeurs singulières.

Cette propriété explique pourquoi Ridge est particulièrement efficace contre la multicolinéarité.

2.3. Régularisation L1 (Lasso)

2.3.1. Définition et motivation

Définition 5.2 (Régularisation L1 - Lasso)

La régularisation Lasso (Least Absolute Shrinkage and Selection Operator) utilise une pénalité L1 :

$$\hat{\theta}_{\text{Lasso}} = \arg \min(\theta) \|y - X\theta\|_2^2 + \lambda \|\theta\|_1$$

où $\|\theta\|_1 = \sum_{j=1}^p |\theta_j|$ est la norme L1.

La différence cruciale avec Ridge est que L1 peut **annuler exactement** certains coefficients, réalisant ainsi une **sélection de variables automatique**.

2.3.2. Propriétés géométriques de L1

La forme géométrique de la contrainte L1 (losange en 2D, polytope en dimension supérieure) favorise les solutions **parses** :

- Les coins du losange correspondent à des solutions où certaines coordonnées sont nulles
- L'optimum a plus de chances d'être atteint en un coin qu'avec la contrainte L2 (cercle)

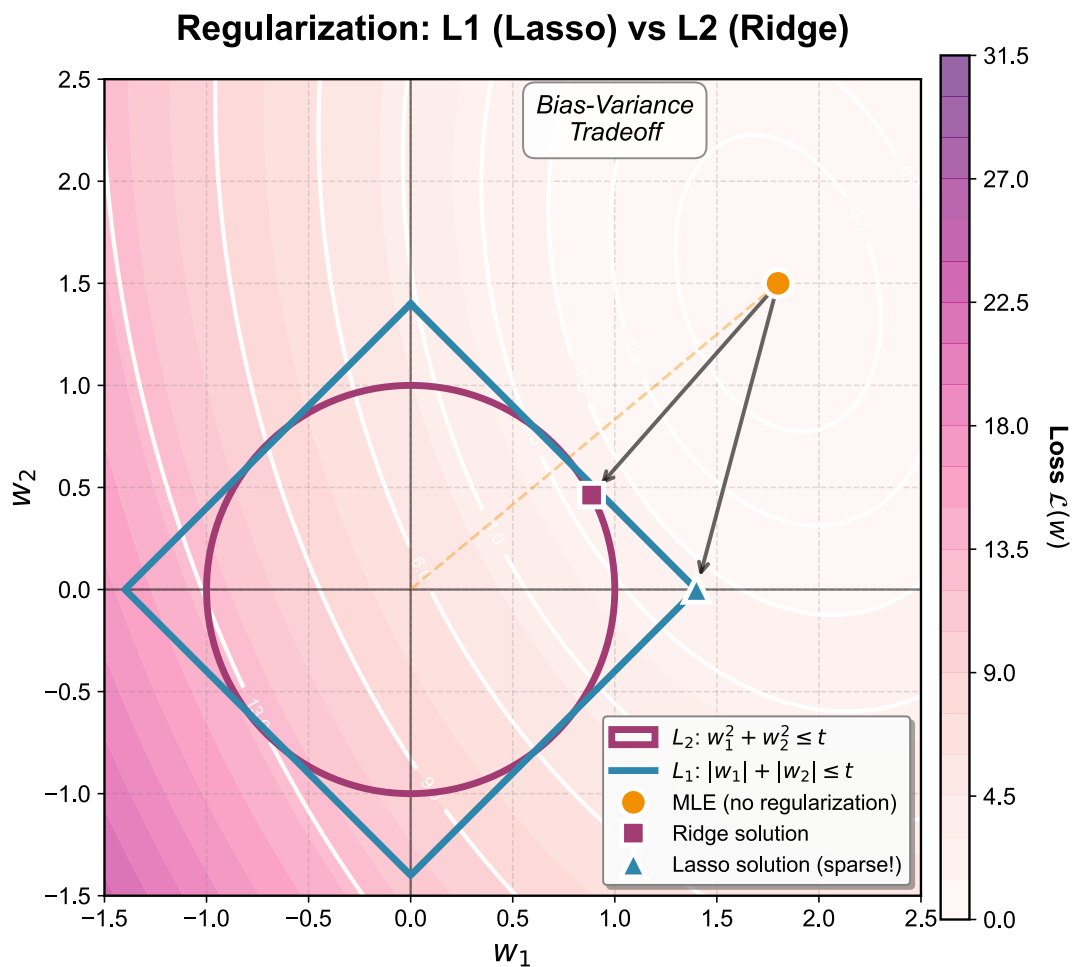


Fig. 1. – Le paysage de notre fonction de perte avec la régularisation L_1 et L_2 . On observe que les solutions L_1 sont plus *parses* que les solutions L_2 .

2.3.3. Lasso path et algorithmes

Définition 5.3 (Lasso path)

Le **lasso path** est la fonction $\lambda \mapsto \hat{\theta}_{\text{Lasso}}(\lambda)$ qui trace l'évolution des coefficients en fonction du paramètre de régularisation.

Propriétés remarquables :

- La fonction est **linéaire par morceaux**
- Les variables entrent/sortent du modèle à des valeurs spécifiques de λ
- Permet de visualiser l'ordre d'importance des variables

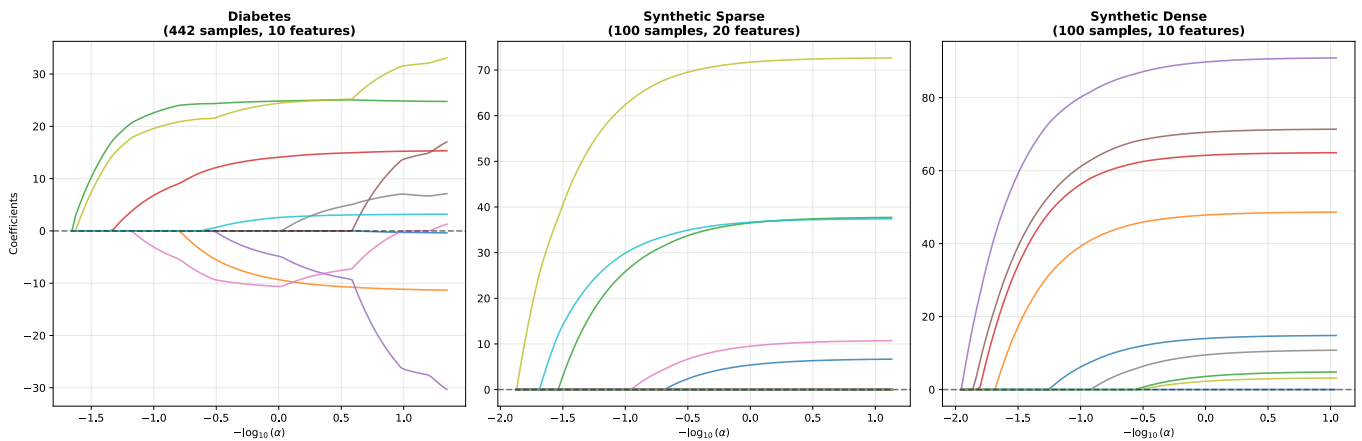


Fig. 2. – Chemins lasso.

2.3.4. Algorithme LARS (Least Angle Regression)

L'algorithme LARS calcule efficacement tout le lasso path :

Algorithme 5.1 (LARS simplifié)

1. **Initialisation** : Tous les coefficients à zéro, résidu $r = y$
2. **Tant que** des variables peuvent être ajoutées :
 - Trouver la variable x_j la plus corrélée avec le résidu :
$$j^* = \arg \max_j |x_j^\top r|$$

- Ajouter x_{j^*} à l'ensemble actif
- Déplacer les coefficients dans la direction « équiangulaire » jusqu'au prochain événement

3. Événements :

- Une nouvelle variable devient maximalement corrélée
- Un coefficient actif atteint zéro (modification Lasso)

2.3.5. Implémentation et visualisation

```
from sklearn.linear_model import Lasso, LassoCV
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# Génération de données avec structure parce
X = np.random.randn(100, 20)
true_coef = np.zeros(20)
true_coef[:5] = [2, -1.5, 1, -0.5, 3] # Seulement 5 coefficients non nuls
y = X @ true_coef + 0.1 * np.random.randn(100)

# Calcul du lasso path
alphas = np.logspace(-4, 1, 50)
coefs = []
for alpha in alphas:
    lasso = Lasso(alpha=alpha, max_iter=1000)
    lasso.fit(X, y)
    coefs.append(lasso.coef_)
```

Exercice 5.1 (Lasso path)

1. Implémentez une fonction qui calcule et visualise le lasso path pour un jeu de données simulé
2. Observez l'ordre dans lequel les variables entrent/sortent du modèle
3. Comparez avec la vraie structure de parcimonie des données

2.4. Elastic Net

2.4.1. Motivation et définition

Elastic Net combine les avantages de Ridge et Lasso pour pallier leurs limitations respectives :

Définition 5.3 (Elastic Net)

$$\hat{\theta}_{\text{EN}} = \arg \min(\theta) \|y - X\theta\|_2^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2$$

Paramétrisation alternative avec $\alpha \in [0, 1]$:

$$\hat{\theta}_{\text{EN}} = \arg \min(\theta) \|y - X\theta\|_2^2 + \lambda[(1 - \alpha)\|\theta\|_2^2 + \alpha \|\theta\|_1]$$

où α contrôle le mélange entre L1 et L2.

2.4.2. Avantages de la combinaison

Méthode	Sélection variables	va-	Groupes corrélés
Ridge	Non		Conserve tous
Lasso	Oui		Sélection arbitraire
Elastic Net	Oui		Sélection groupée

Propriétés d'Elastic Net

1. **Effet de groupement** : Variables corrélées tendent à être sélectionnées/éliminées ensemble
2. **Stabilité** : Plus stable que Lasso face aux perturbations des données

3. **Flexibilité** : α permet d'ajuster le comportement (0 = Ridge, 1 = Lasso)

4. **Parcimonie contrôlée** : Évite la sur-sélection du Lasso

2.4.3. Cas d'usage pratiques

```
from sklearn.linear_model import ElasticNet, ElasticNetCV
```

```
# Elastic Net avec validation croisée pour choisir alpha  
et l1_ratio
```

```
elastic_cv = ElasticNetCV(  
    l1_ratio=[0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1],  
    alphas=100, # Nombre d'alphas testés automatiquement  
    cv=5,  
    random_state=42  
)
```

```
elastic_cv.fit(X_train, y_train)  
print(f"Meilleur alpha: {elastic_cv.alpha_:.4f}")  
print(f"Meilleur l1_ratio: {elastic_cv.l1_ratio:.2f}")
```

Exercice 5.2 (Comparaison des régularisations)

Sur un jeu de données avec des groupes de variables corrélées :

1. Comparez les performances prédictives de Ridge, Lasso et Elastic Net
2. Analysez les patterns de sélection de variables
3. Étudiez l'influence du paramètre α d'Elastic Net

2.5. Validation croisée et choix de λ

2.5.1. Principe de la validation croisée

Le choix du paramètre λ est crucial et se fait typiquement par validation croisée :

Procédure de sélection par CV

1. **Grille de valeurs** : Définir une grille $\Lambda = \{\lambda_1, \dots, \lambda_K\}$
2. **Pour chaque $\lambda_k \in \Lambda$** :
 - Calculer l'erreur de validation croisée $CV(\lambda_k)$
3. **Sélection** : $\hat{\lambda} = \arg \min\{\lambda_k\} CV(\lambda_k)$
4. **Modèle final** : Réentraîner sur toutes les données avec $\hat{\lambda}$

2.5.2. Règles pratiques

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge, Lasso, ElasticNet

# Grilles de paramètres typiques
ridge_params = {'alpha': np.logspace(-4, 4, 50)}
```



```

lasso_params = {'alpha': np.logspace(-4, 1, 50)}

elastic_params = {
    'alpha': np.logspace(-4, 1, 20),
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 0.99]
}

# Validation croisée avec stratification temporelle si
nécessaire
cv_ridge = GridSearchCV(
    Ridge(),
    ridge_params,
    cv=5,
    scoring='neg_mean_squared_error'
)

```

2.6. Applications et extensions

2.6.1. Régularisation en deep learning

Les techniques de régularisation s'étendent naturellement aux réseaux de neurones avec des équivalences mathématiques intéressantes à noter.

2.6.1.1. Weight Decay : L2 en action

Le **weight decay** est littéralement une régularisation L2 appliquée aux paramètres du réseau. Considérons la fonction de coût avec weight decay :

$$L_{\text{total}}(\mathbf{w}) = L_{\text{data}}(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

Lors de la descente de gradient, la mise à jour devient :

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla L_{\text{data}}(\mathbf{w}_t) - \eta \lambda \mathbf{w}_t$$

$$\mathbf{w}_{t+1} = (1 - \eta \lambda) \mathbf{w}_t - \eta \nabla L_{\text{data}}(\mathbf{w}_t)$$

Le terme $(1 - \eta \lambda)$ provoque un **decay** (décroissance) des poids à chaque itération, d'où le nom « weight decay ». Pour $\eta \lambda$ petit, on a approximativement une décroissance exponentielle des poids non mis à jour par les données.

2.6.1.2. Autres techniques modernes

- **Batch normalization** : Régularisation implicite par normalisation et bruit ajouté
- **Dropout** : Régularisation stochastique par masquage aléatoire
- **LayerNorm/GroupNorm** : Variants pour différentes architectures
- **Spectral normalization** : Contrôle de la constante de Lipschitz

2.6.2. Régularisation structurée

Au-delà de L1 et L2, des régularisations plus sophistiquées exploitent la structure des problèmes :

- **Group Lasso** : Sélection de groupes de variables
- **Fused Lasso** : Régularisation spatiale/temporelle
- **Nuclear norm** : Régularisation de rang pour les matrices

2.6.3. Synthèse et recommandations

Guide de choix de la régularisation

Ridge :

- Toutes les variables sont a priori importantes

- Problème de multicolinéarité
- Stabilité numérique requise

Lasso :

- Sélection de variables automatique souhaitée
- Structure parcimonieuse attendue
- Interprétabilité importante

Elastic Net :

- Groupes de variables corrélées
- Compromis entre sélection et stabilité
- $p > n$ (plus de variables que d'observations)

La régularisation constitue un outil fondamental qui permet de transformer des problèmes mal posés en problèmes bien posés, tout en introduisant des a priori utiles sur la structure des solutions recherchées.